

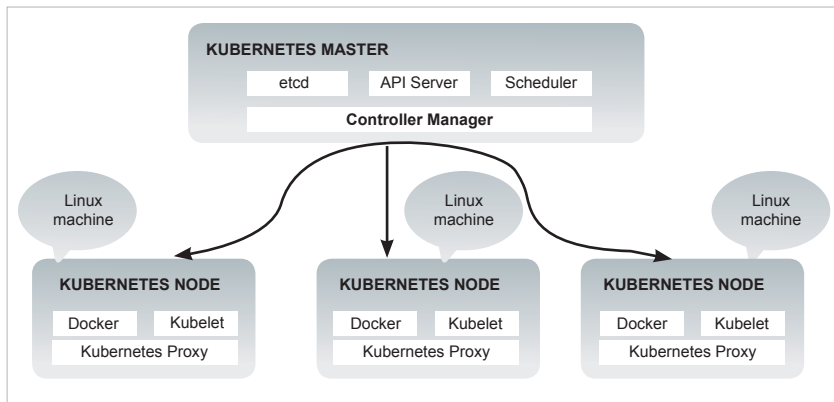


Figure 2: Kubernetes architecture

(PaaS) that creates a hardware layer abstraction for development teams. Nowadays, it supports Amazon Web Services (AWS), Microsoft Azure, the Google Cloud Platform (GCP), or on-premise installations.

Kubernetes is a popular open source platform for container orchestration. It enables developers to easily build containerised applications and services, as well as scale, schedule and monitor those containers. Kubernetes is also highly extensible and portable, which means it can run in a wide range of environments and be used in conjunction with other technologies, such as service meshes.

Figure 2 highlights the Kubernetes architecture.

Kubernetes follows client-server architecture. The master is installed on one machine and the node on separate Linux machines.

Kubernetes: Key components of master

- **etcd:** This stores the configuration information used by every node in the cluster. It is a high availability key value store that can be distributed among multiple nodes. It is a distributed key value store that is accessible to all.
- **API server:** Kubernetes is an API server that provides all the operations on cluster using the API. API server implements an interface,

which means different tools and libraries can readily communicate with it. *Kubeconfig* is a package along with the server side tools. It can be used for communication.

- **Controller manager:** This component is responsible for most of the collectors that regulate the state of a cluster and perform a task. It works towards getting the shared state of a cluster, and then makes changes to bring the current status of the server to the desired state.
- **Scheduler:** This is one of the key components of Kubernetes master. It is a service in master responsible for distributing the workload. It is responsible for tracking the utilisation of working load on cluster nodes, placing the workload on the resources that are available, and then accepting it.

Kubernetes: Key components of node (client)

- **Docker:** This helps in running the encapsulated application containers in a lightweight operating environment.
- **Kubelet service:** This is a small service in each node responsible for relaying information to and from the control plane service. It interacts with the *etcd* store to read configuration details and right values. This communicates with the master component to receive commands and work.

- **Kubernetes proxy service:** This is a proxy service which runs on each node and helps in making services available to the external host. It helps in forwarding the request to correct containers and is capable of performing primitive load balancing.

Deep dive into Apache Mesos

Apache Mesos is an open source software project originally developed at the University of California at Berkeley, but now widely adopted in organisations like Twitter, Uber and PayPal. Mesos' lightweight interface lets it scale easily up to 10,000 nodes (or more). Its APIs support popular languages like Java, C++ and Python, and it also supports out-of-the-box high availability. Unlike Swarm or Kubernetes, however, Mesos only provides management of the cluster; so a number of frameworks have been built on top of it, including Marathon, a 'production-grade' container orchestration platform.

Mesos is a cluster resource manager that targets the principal requirements of:

- Efficient isolation and resource sharing across distributed applications
- Optimal utilisation of resources in cluster
- Support for a wide variety of frameworks
- Hardware heterogeneity
- High scalability, being able to have clusters of tens of thousands of nodes
- Fault tolerance

The architectural components of Apache Mesos are listed below.

- **Mesos master:** This is a mediator between an agent's resources and the frameworks, and undertakes the task of fine-grained resource sharing among frameworks.
- **Mesos agent:** This primarily performs resource management on physical nodes and also runs executors of frameworks.
- **Mesos framework:** It has two